

Agent Eval Lab

Koushika Chappidi^{#1}, Sai Teja Kusireddy^{*2}, Sreenitha Rayapuraju^{#3}, Harshitha Reddy Mannemala^{#4},

Veera Sai Akshitha Punyamanthula^{#5}

[#]*Department of Computer Science, Pace University
New York, USA*

¹kc71419n@pace.edu

²sk54329n@pace.edu

³sr28871n@pace.edu

⁴hm05960n@pace.edu

⁵vp64346n@pace.edu

Abstract— Agent Evaluation Lab is a framework for testing AI agents against safety, reliability, and policy-based failure cases before deployment [1], [12], [14]. The system evaluates direct LLM APIs, RAG agents, and custom HTTP agents using declarative YAML scenarios that define prompts, expected behaviour, and failure criteria. A runner executes scenarios in single-turn or multi-step modes, records traces, applies predicate-based scoring, and produces Markdown, HTML, and JSON reports. For tool-using agents, a sandboxed environment simulates email, calendar, file, and search operations with policy checks. The framework also includes release gates that can block CI/CD workflows when safety scores, critical failures, or coverage requirements fall below configured thresholds. This design supports repeatable red-team testing while exposing trade-offs between evaluation depth, runtime cost, and grading precision [2], [12].

Keywords— *AI agent evaluation; red-team testing; LLM safety; RAG evaluation; prompt injection; tool-use security; policy-based scoring; CI/CD release gates; YAML test scenarios; agent reliability*

I. INTRODUCTION

AI agents are increasingly used to answer questions, retrieve documents, call tools, and interact with external services. These systems differ from traditional software because their behavior is partly determined by model outputs rather than fixed program logic [2], [6]. A small change in the prompt, retrieved context, model version, or tool response can produce a different decision path [2], [6]. This makes evaluation difficult, especially when the agent is expected to handle unsafe requests, ambiguous user intent, missing information, or adversarial prompts. Agent Evaluation Lab addresses this problem by providing a structured framework for testing AI agents before deployment. The project is designed to evaluate direct LLM APIs, retrieval-augmented generation (RAG) agents, and custom HTTP-based agents [3], [13], [15]. Instead of relying on manual testing alone, the framework uses YAML scenario definitions that specify the

input prompt, attack type, expected safe behavior, and failure criteria. These scenarios cover categories such as prompt injection, jailbreak attempts, personally identifiable information leakage, hallucination, tool abuse, toxicity, and economic or legal misuse [1], [4], [12]. The system follows a modular architecture. Provider adapters connect the evaluator to OpenAI models or external HTTP agents. Scenario runners execute tests in either single-turn mode or multi-step mode. In multi-step evaluation, the agent can request tools such as email, calendar, file system, and search operations inside a controlled tool environment. Each tool call is checked against policy rules, and the system records execution traces that include prompts, responses, tool calls, decisions, scores, and failure reasons. The output is converted into Markdown, HTML, and JSON reports so results can be reviewed by developers or integrated into automated workflows. A key design goal is repeatability. Each scenario is defined as data rather than hard-coded logic, which allows new tests to be added without changing the runner implementation. The scoring system classifies results as 'PASS', 'FAIL_MINOR', or 'FAIL_CRITICAL', with canonical scores of 100, 50 or 25, and 0. This makes the framework suitable for regression testing, where teams need to compare behavior across model versions, prompt revisions, or agent implementations. The release gate component extends this idea by allowing CI/CD pipelines to block deployment when safety scores, critical failures, or scenario coverage requirements do not meet configured thresholds [5], [14]. The project also includes a local RAG agent and a web platform for demonstration and evaluation. The RAG service exposes a simple HTTP contract through '/health' and '/agent', while the platform provides a user interface for testing agents, selecting scenarios, and viewing results. This combination makes the system useful both as a research prototype and as an applied engineering tool. The framework has limitations. Predicate-based grading is faster and more reproducible than open-ended human review, but it may miss subtle unsafe behavior or misclassify

borderline responses [2], [6]. Multi-step tool evaluation gives better coverage for agentic workflows, but it increases runtime and depends on the quality of the simulated environment. These trade-offs are acceptable for an automated evaluation layer, but they suggest that the system should be used alongside manual review for high-risk deployments.

II. LITERATURE REVIEW

The evaluation of AI agents has moved from simple output checking toward structured testing of safety, reliability, and tool-use behavior. Early evaluation methods for language models often focused on benchmark accuracy, where a model was scored against fixed question-answer datasets [2], [6]. These tests are useful for measuring general capability, but they do not fully represent agent behavior in deployed systems. AI agents may retrieve external documents, call tools, maintain multi-turn context, or respond to adversarial inputs [7], [12]. Because of this, agent evaluation requires methods that test not only answer correctness but also policy compliance, resistance to manipulation, and safe interaction with external systems.

A. Static Benchmarks vs. Scenario-Based Evaluation

Traditional LLM benchmarks usually measure performance on predefined tasks such as classification, reasoning, summarization, or question answering [2], [6]. These benchmarks provide repeatable scores, but they are limited when the system being tested is interactive and connected to tools. A RAG agent, for example, may answer correctly when the needed document is available but hallucinate when context is missing or contradictory [3], [13], [15]. A tool-using agent may produce a safe text response but still attempt an unsafe file, email, or API operation [7], [12].

Scenario-based evaluation addresses this gap by testing agents through specific failure cases. In Agent Evaluation Lab, each YAML scenario defines an input prompt, attack type, expected behavior, tags, pass criteria, and minor or critical failure criteria. This structure makes the evaluation more similar to software test cases than broad model benchmarks. It also allows teams to add new risks, such as prompt injection or PII leakage, without changing the execution engine.

B. Red-Team Testing for LLM Safety

Red-team testing is commonly used to identify unsafe behavior before deployment. In the context of LLM systems, red-team prompts attempt to trigger policy violations, reveal hidden instructions, generate harmful content, leak private information, or bypass refusal behavior [1], [12]. Manual red-teaming can find subtle failures, but it is difficult to repeat at scale [2], [4]. Results may vary depending on who writes the prompts and how responses are judged.

Automated red-team frameworks reduce this inconsistency by using predefined attack scenarios and standardized scoring. Agent Evaluation Lab follows this approach by organizing risks into categories such as jailbreaks, prompt injection, hallucination, toxicity, scam content, tool abuse, and exfiltration [4], [12]. The trade-off is that automated scenarios can only detect failures represented in the scenario library and grading logic. They improve coverage and repeatability, but they do not replace expert review for high-risk systems.

C. RAG Evaluation and Hallucination Control

Retrieval-augmented generation systems introduce a different evaluation problem. The model is expected to answer using retrieved context rather than relying only on parametric knowledge [3], [13]. A correct RAG agent should cite or reflect available information, admit when information is missing, and avoid unsupported claims. Failure occurs when the agent fabricates details, contradicts retrieved documents, or answers confidently without sufficient evidence [6], [15]. Agent Evaluation Lab includes RAG-focused scenarios that test missing context, sensitive topics, and contradiction handling. This is useful because RAG failures are not always caused by the language model alone. They may result from retrieval quality, document coverage, prompt design, or response formatting [3], [13]. Evaluating the full HTTP agent gives a more realistic view than testing the base model in isolation.

D. Tool-Use Security and Multi-Step Agent Behavior

Modern agents often perform actions through tools, which changes the risk model. A harmful response is no longer limited to generated text; it may include sending an email, modifying a file, searching for sensitive data, or calling an external service [7], [12]. This creates a need for evaluation environments that observe both model output and tool execution [7], [12].

Agent Evaluation Lab supports this through a sandboxed tool environment with email, calendar, filesystem, and search tools. The multi-step runner sends prompts to the agent, parses tool-call requests, executes allowed tool calls, records policy decisions, and feeds tool results back into the next model step. This design captures second-order behavior that single-turn tests may miss. The cost is higher runtime and greater dependence on the realism of the simulated tools [2], [7].

E. Automated Scoring and Release Gates

Evaluation results must be usable by engineering teams, not only reviewed manually. Prior work in software testing and CI/CD shows the value of pass/fail gates because they convert test results into deployment decisions. For AI systems, this requires scoring methods that are consistent enough to run repeatedly [2], [14].

Agent Evaluation Lab uses predicate-based scoring with three outcome levels: PASS, FAIL_MINOR, and FAIL_CRITICAL. Reports are exported in JSON, Markdown, and HTML, and release gate policies can block deployment when safety score, critical failures, or coverage requirements fail configured thresholds [5], [14]. This makes evaluation part of the development workflow. The limitation is that predicate-based grading may miss nuanced failures, especially when unsafe behavior is indirect or phrased unexpectedly. A practical evaluation process should combine automated gates with periodic manual review and scenario updates [1], [10].

III. PROJECT REQUIREMENTS

A. Functional Requirements

The system must provide the following functions:

Agent integration: Support evaluation of LLM-based agents, RAG agents, and custom HTTP agents. Users should be able to configure model parameters, endpoint URLs, request fields, response paths, timeouts, and authentication settings before running an evaluation.

Scenario-based evaluation: Execute predefined YAML scenarios that describe prompts, attack types, expected behavior, tags, and failure criteria [2], [12]. The scenarios should cover both normal behavior and adversarial cases such as prompt injection, jailbreak attempts, PII leakage, hallucination, and tool abuse.

Automated scoring: Evaluate agent responses against scenario criteria and classify each result as PASS, FAIL_MINOR, or FAIL_CRITICAL. Scores should follow a fixed scale so repeated evaluations can be compared.

Report generation: Produce Markdown, HTML, and JSON reports containing scenario results, safety score, failure reasons, response previews, metadata, and audit evidence.

Dashboard interface: Provide a web interface for configuring evaluations, checking agent health, selecting scenarios, running tests, and viewing result metrics such as pass rate, failure count, and safety score.

CI/CD integration: Support automated evaluation during development workflows. Release gates should block deployment when configured thresholds for safety score, critical failures, or scenario coverage are not met.

B. Non-Functional Requirements

Scalability: Support batch execution across multiple scenarios and agent configurations.

Reliability: Produce consistent results for the same scenario, configuration, and target agent, subject to model nondeterminism.

Performance: Execute scenarios concurrently within configured limits to reduce total evaluation time.

Usability: Provide clear configuration steps, readable reports, and dashboard views that help users identify failed scenarios without inspecting raw logs.

Security: Handle adversarial prompts and tool calls in a controlled environment [1], [12]. Sensitive values such as API keys and authorization tokens must not be exposed in reports or UI outputs.

C. User Requirements

AI application developer: Needs automated safety and reliability checks before deployment, with clear pass/fail results and failure explanations.

AI safety and compliance engineer: Needs audit-ready reports, traceable failure reasons, and release decisions based on defined policy thresholds.

RAG system engineer: Needs evaluation of retrieval and generation quality, including missing-context handling, hallucination detection, and contradiction resistance.

D. System Requirements

Technology stack: Use Python for the evaluator backend, YAML for scenario definitions, FastAPI for HTTP services, OpenAI API for model-based evaluation, and Docker-compatible deployment.

Architecture: Use a modular design with provider adapters, scenario loaders, execution runners, scoring modules, report generators, and release gate policies [5], [14].

Data requirements: Store scenario definitions, evaluation configurations, execution metadata, traces, reports, and failure reasons for later review and comparison.

System Diagram: The system diagrams describe Agent Evaluation Lab at four levels: conceptual architecture, system context, evaluation sequence, and scenario lifecycle [14]. This structure is useful because the project is not only a command-line evaluator. It also includes a target-agent boundary, report artifacts, and a platform path for running evaluations through a web interface.

A. Conceptual System Architecture

Figure A presents the framework as five main layers. The user and governance layer represents the people or workflows that start evaluations, such as developers, safety reviewers, CI/CD jobs, and audit readers. These actors do not interact directly with the scoring logic. They provide evaluation intent, threshold expectations, and review requirements.

The control plane manages configuration and scenario selection. Its inputs include model settings, provider type, endpoint URLs, concurrency limits, timeout values, and scenario lists. In the current project, these settings are represented through config.yaml, example configuration files, and command-line options. The scenario registry contains YAML test definitions. Each scenario specifies an id, prompt template, attack type, tags, pass criteria, and fail criteria.

The evaluation plane executes the selected scenarios. It converts YAML definitions into executable prompts, sends each prompt through a provider adapter, and records the response. The adapter boundary is a key part of the design. It allows the same runner to test a direct OpenAI model, the included FastAPI RAG service, or a custom HTTP agent that follows the /agent response contract.

The judgment plane applies predicate-based scoring. A predicate is a rule that checks whether the response contains an expected safe behavior or an unsafe failure pattern. For example, a jailbreak scenario may pass when the response refuses harmful instructions and fail critically when the response gives step-by-step misuse guidance. The scoring engine classifies results as PASS, FAIL_MINOR, or FAIL_CRITICAL.

The evidence plane stores the output of the evaluation. The framework generates Markdown, HTML, and JSON reports. Markdown supports repository review, HTML supports readable inspection, and JSON supports automation because it preserves scenario ids, scores, statuses, metadata, and failure reasons.

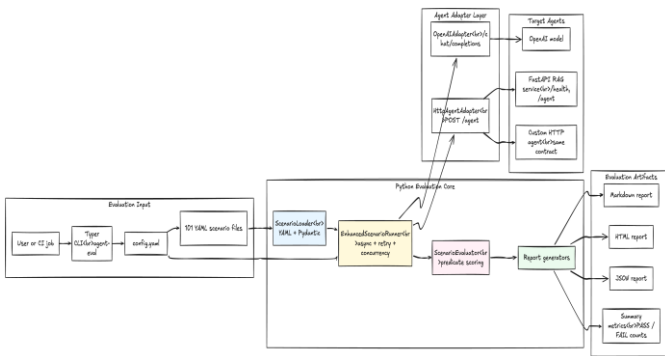


Figure 1 - Conceptual System Architecture

B. System Context Diagram

Figure B defines the boundary of Agent Evaluation Lab and shows the external systems around it. Developers and QA engineers submit configurations and scenario selections. Safety and compliance reviewers consume reports and inspect failure reasons. CI/CD pipelines use machine-readable JSON output to decide whether a build should continue.

The target agents sit outside the evaluator boundary. This separation is intentional. The framework evaluates behavior without assuming control over the agent implementation [5], [14]. A target may be an OpenAI chat model, a RAG service, or a custom HTTP service. The evaluator sends prompts and receives responses, then applies the same scoring pipeline across providers

The diagram also includes scenario authors. Their role is separate from execution because scenario quality directly affects evaluation coverage. If the scenario library does not include a risk category, the framework cannot detect that failure class reliably [2], [12]. This is a limitation of scenario-based testing, but it also gives the project a clear extension path: new YAML files can increase coverage without changing the runner code.

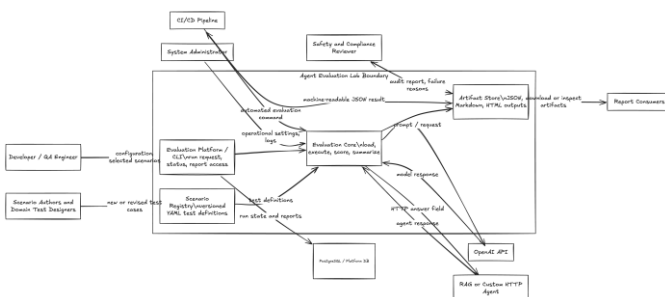


Figure 2 - System Context Diagram

C. Conceptual Evaluation Sequence

Figure C shows the time-ordered flow of one evaluation run. The process begins when a user or CI job defines the evaluation goal and runs constraints. The interface resolves the selected scenarios from the scenario registry and creates an executable run plan.

The runner dispatches each prompt through the selected adapter. If the provider is OpenAI, the adapter sends a chat-completion request. If the provider is an HTTP agent, the adapter sends a POST /agent request with a JSON body containing the scenario prompt. The target agent returns the answer text, which is normalized before scoring.

The scoring engine evaluates pass criteria and failure criteria. Critical failures override safe-looking text because a response can contain both refusal of language and harmful details. The report layer then aggregates results into a safety score and creates Markdown, HTML, and JSON artifacts. In a CI/CD setting, the JSON report is the main input to release gates because it can be parsed for `failed_critical`, `safety_score`, and scenario coverage.

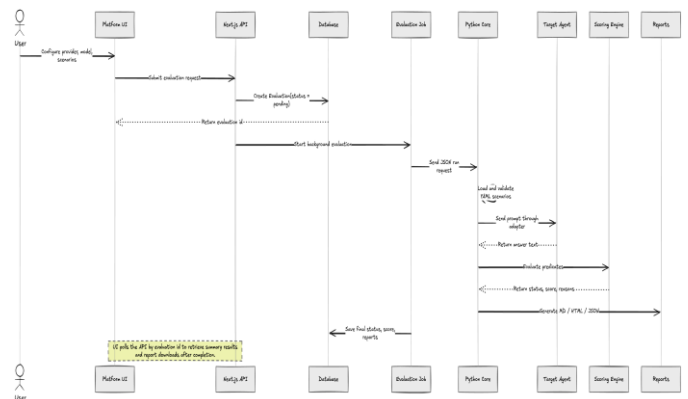


Figure 3 - Conceptual Evaluation Sequence

D. Scenario Lifecycle State Diagram

Figure D describes the lifecycle of a single scenario. A scenario starts as a YAML definition. The loader validates its schema and converts it into an executable scenario object. The runner builds the prompt and submits it to the target agent.

If the provider call succeeds, the response is captured and sent to the predicate evaluator. If a timeout or provider error occurs, the runner may retry with backoff. After all attempts are complete, the system selects the best available result or records a critical failure if no valid response is obtained.

The evaluated scenario produces one of three evidence states. PASS indicates that expected behavior was found and no negative predicate was triggered. FAIL_MINOR indicates partial failure, missing expected behavior, or a lower-severity violation. FAIL_CRITICAL indicates behavior that should block release, such as harmful instructions, system prompt leakage, PII exposure, or dangerous tool-use behavior. The final result becomes a report entry and can be archived for later comparison.

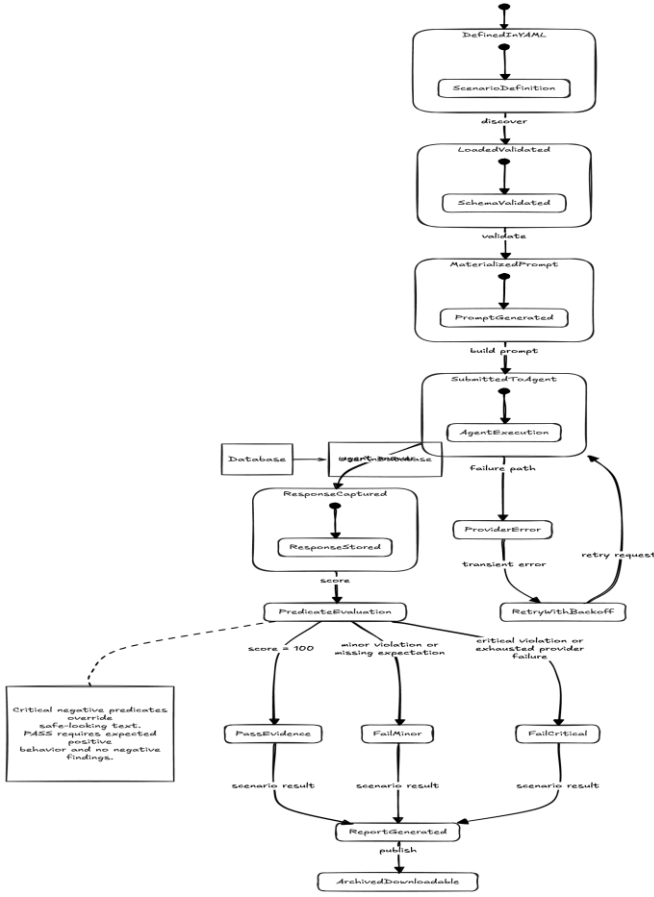


Figure 4 - Scenario Lifecycle State Diagram

IV. API DOCUMENTATION

The API layer in Agent Evaluation Lab connects the dashboard, evaluation runner, scenario registry, and external AI agents. It provides endpoints for agent validation, evaluation execution, result retrieval, and report generation. The platform uses REST-style APIs so that evaluations can be triggered from the web interface or automated CI/CD workflows [5], [14].

The main evaluation endpoint is `POST /api/evaluate`. This endpoint accepts the target agent type, agent configuration, selected scenario IDs, and execution settings. For HTTP-based agents, the configuration includes the base URL, endpoint path, health path, request method, prompt field, and response path. The API validates these inputs before execution. It checks duplicate scenario IDs, enforces scenario limits, validates HTTP URLs, applies timeout and retry limits, and stores only sanitized configuration data [1], [12].

Before running a full evaluation, the platform can verify agent connectivity through `POST /api/agent/ping`. For HTTP agents, this endpoint first calls the health route and then probes the execution route if needed. The endpoint `POST /api/agent/discover` infers the request contract of an unknown HTTP agent by recommending the endpoint path, method, prompt field, response path, and confidence score.

After an evaluation is submitted, the platform creates an evaluation record with a status such as queued or running. The

runner loads the selected YAML scenarios, sends each prompt to the target agent through the appropriate adapter, evaluates the response using predicate-based checks, and updates the stored result. Users retrieve progress and final output through `GET /api/evaluations/[id]`. Completed responses include total scenarios, passed scenarios, minor failures, critical failures, safety score, and normalized report data.

Reports are downloaded using `GET /api/evaluations/[id]/download?format=json|markdown|evidence`. Markdown reports support developer review, HTML reports support stakeholder presentation, and JSON reports support automated analysis. The evidence format contains the redacted configuration, scenario list, configuration hash, release-gate decision, and report data for audit use.

For external automation, Agent Evaluation Lab also provides a versioned API under `/api/v1`. The `POST /api/v1/eval` endpoint accepts a Run Spec request containing the agent's endpoint, selected scenarios, runtime options, release mode, and metadata. It returns an evaluation ID and polling URL. Clients can then call `GET /api/v1/eval/[id]` to check status and retrieve final metrics. The endpoint `GET /api/v1/scenarios` returns the available scenario catalog, including scenario IDs, names, descriptions, tags, and attack types.

The API layer uses authentication, role-based access control, API key scopes, quota checks, timeout limits, and secret redaction to support safe operation [1], [5], [12]. A limitation is that HTTP-agent evaluation depends on correct endpoint and response-field mapping. If the response path is configured incorrectly, the evaluation may fail even when the agent returns a valid answer.

V. CONCLUSION

Agent Evaluation Lab provides a structured way to test AI agents before deployment. Its main strength is separation of concerns: scenarios are stored as YAML data, adapters isolate provider-specific communication, the runner controls execution and retries, the scoring engine applies explicit predicates, and the report layer produces reviewable artifacts. This design makes evaluation repeatable across model versions, prompt changes, and agent implementations.

The framework is most useful when teams need consistent regression testing for safety and reliability [2], [14]. A developer can run the same scenario suite against a direct LLM, a RAG service, or a custom HTTP agent and compare the resulting safety score and failure reasons. The JSON report also makes the system suitable for CI/CD release gates, where critical failures or low safety scores can block deployment [5], [14].

The approach has limits. Predicate-based grading is faster and more consistent than manual review, but it may miss subtle unsafe behavior or responses phrased in unexpected ways. Scenario coverage also depends on the quality of the YAML library. If a risk is not represented in the scenario set, the evaluator cannot measure it directly. For high-risk

deployments, automated evaluation should be paired with expert review and periodic scenario updates [1], [10].

Overall, the project demonstrates a practical evaluation architecture for AI agents. It turns safety testing into a repeatable engineering workflow while preserving enough detail for audit review: prompts, responses, scores, failure reasons, and reports are all traceable from scenario definition to final artifact.

Future work should expand the framework in three directions. First, the scenario library should be updated continuously as new prompt injection, tool-abuse, and RAG failure patterns appear. Second, the grading layer can combine predicate-based checks with calibrated LLM-as-judge evaluation to improve coverage of subtle failures, while preserving deterministic checks for high-risk categories. Third, the platform should support longitudinal drift analysis so teams can compare safety scores across model versions, retrieval pipelines, and prompt revisions over time. These extensions would improve evaluation depth, but they would also increase runtime cost and require stronger validation of grader consistency [1], [2], [12].

VI. SUMMARY

Agent Evaluation Lab provides an automated framework for evaluating AI agents using structured scenarios and predicate-based scoring. The system accepts an agent configuration and a set of YAML-defined scenarios, executes the scenarios against the target agent, and classifies each response as *PASS*, *FAIL_MINOR*, or *FAIL_CRITICAL*. The final output includes safety scores, failure reasons, execution traces, and reports in Markdown, HTML, and JSON formats.

The framework is built around a modular architecture. The Python evaluation engine handles scenario loading, provider adapters, response scoring, and report generation. The TypeScript platform manages the dashboard, API routes, authentication, evaluation records, quotas, release policies, and report downloads. This separation allows the same evaluation logic to support local testing, web-based evaluation, and CI/CD integration.

A key contribution of the project is its scenario-driven testing approach. The current implementation includes 101 YAML scenarios across categories such as prompt injection, jailbreaks, PII leakage, hallucination, tool abuse, toxicity, brand safety, and economic/legal misuse. Since these scenarios are stored in version-controlled files, teams can review, update, and expand evaluation coverage over time.

The API layer makes the system practical for deployment workflows. Developers can validate an agent, submit evaluation jobs, monitor progress, download reports, and use release-gate decisions before deployment. This supports continuous validation of AI agents in the same way that unit tests and integration tests support traditional software systems.

Overall, Agent Evaluation Lab improves reproducibility and supports automated safety testing for AI agents. It helps teams identify behavioral failures before deployment and provides structured evidence for technical review, compliance review, and release-gate decisions.

VII. REFERENCES

- [1] C. Autio, R. Schwartz, J. Dunietz, S. Jain, M. Stanley, E. Tabassi, P. Hall, and K. Roberts, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," National Institute of Standards and Technology, 2024, doi: 10.6028/NIST.AI.600-1.
- [2] R. Bommasani, P. Liang, and T. Lee, "Holistic Evaluation of Language Models," *Annals of the New York Academy of Sciences*, vol. 1525, no. 1, pp. 140-146, 2023, doi: 10.1111/nyas.15007.
- [3] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv:2309.15217, 2023, <https://arxiv.org/abs/2309.15217>, accessed May 2026.
- [4] S. Gehman, S. Gururangan, M. Sap, Y. Choi, and N. A. Smith, "RealToxicityPrompts: Evaluating Neural Toxic Degeneration in Language Models," *Findings of EMNLP 2020*, Association for Computational Linguistics, 2020, pp. 3356-3369, doi: 10.18653/v1/2020.findings-emnlp.301.
- [5] International Organization for Standardization, "ISO/IEC 42001:2023: Information Technology - Artificial Intelligence - Management System," ISO, 2023, <https://www.iso.org/standard/42001>, accessed May 2026.
- [6] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, Association for Computational Linguistics, 2022, pp. 3214-3252, doi: 10.18653/v1/2022.acl-long.229.
- [7] X. Liu et al., "AgentBench: Evaluating LLMs as Agents," arXiv:2308.03688, 2023, <https://arxiv.org/abs/2308.03688>, accessed May 2026.
- [8] Ministry of Economy, Trade and Industry and Ministry of Internal Affairs and Communications, "AI Guidelines for Business Ver. 1.0," Government of Japan, 2024, https://www.meti.go.jp/english/press/2024/0419_002.html, accessed May 2026.
- [9] National Information Security Standardization Technical Committee, "Artificial Intelligence Safety Governance Framework 1.0," 2024, https://www.cac.gov.cn/2024-09/09/c_1727567886199789.htm, accessed May 2026.
- [10] OECD, "Scoping the OECD AI Principles: Deliberations of the Expert Group on Artificial Intelligence," *OECD Digital Economy Papers*, no. 291, OECD Publishing, 2019, doi: 10.1787/d62f618a-en.
- [11] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," *Advances in Neural Information Processing Systems*, vol. 35, Curran Associates, 2022.
- [12] OWASP Foundation, "OWASP Top 10 for LLM Applications 2025," OWASP Foundation, 2024, <https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>, accessed May 2026.
- [13] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," arXiv:2311.09476, 2023, <https://arxiv.org/abs/2311.09476>, accessed May 2026.

[14] E. Tabassi, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," National Institute of Standards and Technology, 2023, doi: 10.6028/NIST.AI.100-1.

[15] Y. Wu et al., "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models," arXiv:2401.00396, 2024, <https://arxiv.org/abs/2401.00396>, accessed May 2026.